

HealthTech

Documento Técnico Interno

Guía de incorporación para nuevos miembros del equipo

Proyecto Informática II

Equipo: Leonardo Borgo · Matías Crego

Versión: entrega de avance — Junio 2026

1. ¿Qué es HealthTech?

HealthTech es una aplicación de orientación médica. El usuario describe sus síntomas y la app le indica si puede manejarlo desde casa, si conviene pedir un turno médico o si necesita dirigirse a urgencias. No diagnostica enfermedades: guía al usuario hacia la decisión correcta, de forma simple y rápida.

Internamente, el sistema está organizado en partes que trabajan juntas: lo que ve el usuario (la interfaz o frontend), lo que procesa la información por detrás (el backend) y, en el futuro, una base de datos para persistir la información de cada usuario.

2. Arquitectura general del sistema

El proyecto se organiza en tres capas:

2.1 Frontend (lo que ve el usuario)

Es la pantalla, los botones, los colores, el chatbot, el mapa y la guía de RCP. El usuario interactúa con esta capa directamente desde el navegador. Actualmente está implementada con HTML5, CSS3 y JavaScript puro (sin frameworks), y funciona abriendo el archivo `index.html`, sin necesidad de instalar nada.

2.2 Backend (lo que procesa la información)

Es el «cerebro» de la aplicación. Contiene la lógica del triaje y el registro de fichas médicas. Está escrito en Python. En esta entrega, el backend funciona como scripts de consola (CLI) independientes, que sirven para probar y validar la lógica antes de conectarla con el frontend mediante una API REST.

2.3 Base de datos (próxima entrega)

En esta etapa no se utiliza una base de datos. En la entrega final se incorporará el almacenamiento de fichas médicas, historial de síntomas y registros de evolución, protegidos con encriptación para garantizar la privacidad del usuario.

3. Stack tecnológico

Componente	Tecnología	Estado actual
Interfaz (Frontend)	HTML5, CSS3, JavaScript vanilla	Implementado
Mapas y geolocalización	Leaflet.js + OpenStreetMap (Nominatim y Overpass API)	Implementado

Lógica de triaje y fichas	Python 3.10+ (triage.py, usuarios.py, utils.py)	Implementado (CLI)
API / Servidor	Flask (Python)	Próxima entrega
Diseño UI/UX	Figma	En uso continuo
Control de versiones	Git / GitHub	En uso continuo
Despliegue / Hosting	A definir (ej. Vercel)	Planificado

4. Estructura del proyecto (entrega de avance)

Para esta entrega, el proyecto funciona completamente en local, sin servidor. La estructura de carpetas actual es la siguiente:

```
HealthTech/  
  ■■■ frontend/  
    ■ ■■■ index.html  
    ■ ■■■ app.js  
    ■ ■■■ style.css  
    ■ ■■■ logo_png.png  
  ■■■ backend/  
    ■ ■■■ triage.py    → Lógica de evaluación de riesgo (CLI)  
    ■ ■■■ usuarios.py → Registro de fichas médicas (CLI)  
    ■ ■■■ utils.py    → Funciones auxiliares compartidas  
  ■■■ docs/  
    ■■■ HealthTech_Ficha-Tecnica.pdf  
    ■■■ HealthTech_Stakeholders.pdf
```

5. Flujo de trabajo del equipo

- Antes de empezar una tarea, revisás en qué rama estás y creás una nueva con un nombre descriptivo. Ejemplo: *feature/hospitales-overpass*.
- Mientras trabajás, hacés commits frecuentes con mensajes claros que expliquen qué cambiaste.
- Cuando terminás, subís tu rama a GitHub y abrís un Pull Request para que otro miembro lo revise.
- Una vez aprobado, se une a **main** y tu cambio ya forma parte del proyecto oficial.

Nunca se trabaja directamente sobre **main**. Esa rama siempre tiene que tener el código que funciona.

6. Diseño visual: cómo usamos Figma

En Figma están diseñadas las pantallas antes de ser programadas. Si vas a desarrollar una nueva pantalla o componente: buscás el diseño en Figma, revisás colores, tamaños y comportamiento esperado, y recién entonces empezás a programarlo.

El sistema de colores tiene una lógica funcional: **verde y azul** para estados normales o leves, **amarillo** para situaciones que requieren atención próxima, **rojo** para emergencias. El color que muestra la app es parte del resultado del triaje.

7. Lógica del triaje (cómo piensa la app)

El corazón del sistema está en **triage.py**. La lógica funciona por puntaje:

- El usuario indica el nivel de dolor (escala 1-10) y la temperatura corporal.
- Según el dolor: 1-3 suma 5 pts · 4-7 suma 10 pts · 8-10 suma 15 pts.
- Según la temperatura: <37 °C suma 0 · 37-38 suma 5 · 38-39 suma 10 · ≥39 suma 15.
- Resultado: **LEVE** (<10 pts) · **MODERADO** (10-20 pts) · **URGENTE** (>20 pts).

Si vas a modificar esta parte del código, entendé bien la lógica primero: un cambio en los valores de corte afecta directamente lo que el usuario ve como resultado. Están basados en una simplificación de protocolos de triaje reales y se ajustarán con criterios médicos más precisos en futuras entregas.

8. Asistente conversacional (Chatbot)

El chatbot funciona del lado del frontend, en **app.js**. Cada categoría de síntoma tiene asociada una lista de palabras clave ("keys") y un mensaje de orientación. Cuando el usuario escribe un mensaje, el sistema lo pasa a minúsculas y revisa si contiene alguna de esas palabras; si hay coincidencia, responde con el mensaje de esa categoría.

En esta entrega el chatbot cubre más de 45 categorías (dolor de pecho, fiebre, alergias, convulsiones, quemaduras, picaduras, salud mental, embarazo, intoxicaciones, RCP, primeros auxilios, entre muchas otras) con más de 200 palabras clave en total.

9. Módulo de mapas y hospitales

La sección Hospitales usa Leaflet.js con OpenStreetMap. Los hospitales ya no están escritos a mano: se buscan en tiempo real.

- **Geolocalización:** si el usuario lo permite, se centra el mapa en su ubicación actual.

- **Búsqueda de direcciones:** se usa la API de Nominatim para convertir una dirección en coordenadas.
- **Hospitales cercanos:** la API de Overpass obtiene hospitales y centros de salud reales dentro de 5 km.

Cada hospital se muestra como un marcador, con un popup que incluye su nombre y, si está disponible en OpenStreetMap, un botón para llamar por teléfono.

10. Datos y privacidad

- Nunca subas datos reales de usuarios al repositorio de GitHub, ni en archivos de prueba.
- Para testear el sistema, usá siempre datos ficticios.
- Si encontrás información sensible expuesta en el código, avisá de inmediato antes de hacer cualquier commit.

11. Estado actual del desarrollo

Esta entrega de avance incluye una aplicación 100% funcional en el navegador, sin necesidad de levantar un servidor:

- Chatbot de orientación con más de 45 categorías de síntomas.
- Mapa interactivo con hospitales reales cercanos a la ubicación del usuario.
- Guía paso a paso de RCP.
- Botón de llamada directa al 911.
- Lógica de triaje y registro de fichas médicas validadas por consola en Python.

12. Roadmap técnico — Próximas implementaciones

A partir de esta entrega, el equipo identificó los siguientes puntos como trabajo a futuro, clasificados por prioridad:

Implementación	Prioridad	Descripción
Despliegue en servidor / web / app	CONFIRMADO	Conectar el backend Flask con el frontend mediante una API REST y publicar la app en una plataforma de hosting (por ejemplo Vercel), accesible desde una URL pública y eventualmente empaquetable como app móvil.

Mejoras visuales	CONFIRMADO	Revisión integral del diseño en Figma: paleta de colores, tipografía, iconografía, animaciones y adaptación responsive para celular, tablet y escritorio. Incluye mejoras de accesibilidad para adultos mayores.
Expansión del chatbot	CONFIRMADO	Incorporar muchas más palabras clave y categorías de síntomas, mejorar el algoritmo de coincidencia (más tolerante a errores de tipeo y sinónimos) y agregar nuevas funcionalidades conversacionales.
Información ampliada de hospitales	CONFIRMADO	Al pasar el mouse (o tocar) sobre un hospital en el mapa, mostrar una tarjeta completa: dirección, teléfono, especialidades, y si el centro está abierto o cerrado en ese momento, usando horarios disponibles en OpenStreetMap.
Integración con IA	EXPLORATORIO	Idea en evaluación: conectar el chatbot con un modelo de lenguaje (IA) para entender síntomas en lenguaje natural en vez de palabras clave fijas, y/o asistir en la priorización del triaje. Se evaluará su viabilidad y costo antes de comprometerse a implementarla.

Los primeros cuatro puntos están planificados para la entrega final. La integración con IA es una idea que se evaluará en paralelo: si resulta viable dentro de los tiempos y recursos del proyecto, se incorporará como una mejora adicional.

13. Primeros pasos al incorporarte

- Pedile acceso al repositorio de GitHub y al archivo de Figma a algún miembro del equipo.
- Leé el README.md del repositorio: ahí están los pasos para correr el proyecto en tu computadora.
- Instalá Python 3.10 o superior si todavía no lo tenés.
- Cloná el repositorio: `git clone [url]`.
- Abrí **frontend/index.html** en el navegador para ver la aplicación funcionando.
- Revisá los diseños en Figma para entender visualmente cómo se ve la app.
- Antes de tocar cualquier archivo, creá tu propia rama de trabajo.

14. Contacto y dudas

Si algo no queda claro en este documento, consultale directamente a algún compañero del equipo. En proyectos como este, preguntar antes de asumir siempre ahorra tiempo. El código también tiene

comentarios en los puntos más importantes para ayudarte a entender qué hace cada parte.